

Đặc điểm của Bagging và Pasting



Hàm tổng hợp

Chế độ thống kê (đối với phân loại) hoặc giá trị trung bình (đối với hồi quy).



Bias / Variance

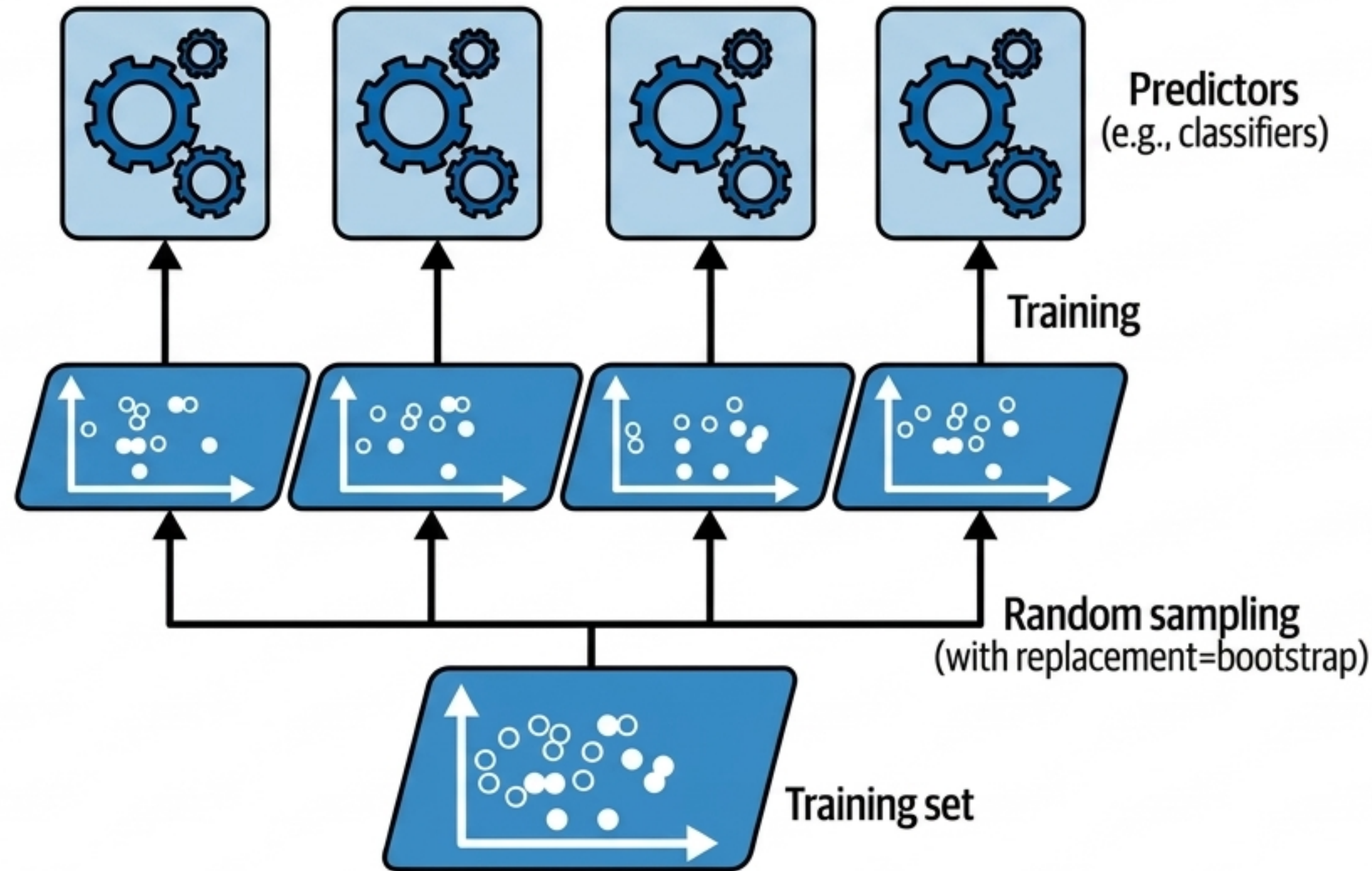
Bộ dự đoán lẻ: độ lệch (bias) cao hơn.
Kết hợp lại: độ lệch tương tự nhưng phương sai (variance) thấp hơn rất nhiều.



Khả năng mở rộng

Mở rộng tốt. Huấn luyện song song dễ dàng trên đa lõi CPU hoặc cụm máy chủ.

Quá trình lấy mẫu Bagging/Pasting



Hình 7-4. Túi hóa và Dán nhãn trên các mẫu ngẫu nhiên.

Bagging và Pasting trong Scikit-Learn

Scikit-Learn cung cấp lớp `BaggingClassifier` (hoặc `BaggingRegressor`).



n_estimators

Số lượng bộ dự đoán (ví dụ: 500 cây).



max_samples

Kích thước tập mẫu ngẫu nhiên cho mỗi bộ.



bootstrap=True

Sử dụng Bagging (False cho Pasting).



n_jobs=-1

Sử dụng tất cả các lõi CPU để tăng tốc.

Mã nguồn: BaggingClassifier



Huấn luyện tập hợp gồm 500 cây quyết định bằng phương pháp Bagging.

```
1 from sklearn.ensemble import BaggingClassifier
2 from sklearn.tree import DecisionTreeClassifier
3
4 bag_clf = BaggingClassifier(
5     DecisionTreeClassifier(),
6     n_estimators=500,
7     max_samples=100,
8     bootstrap=True, # Doi thanh False de dung Pasting
9     n_jobs=-1,
10    random_state=42
11 )
12 bag_clf.fit(X_train, y_train)
```

Phân tích kết quả của Bagging



Đa dạng hơn trong các tập con huấn luyện (độ lệch nhỏ hơn một chút so với Pasting).



Sự đa dạng giúp các bộ dự đoán ít tương quan hơn.



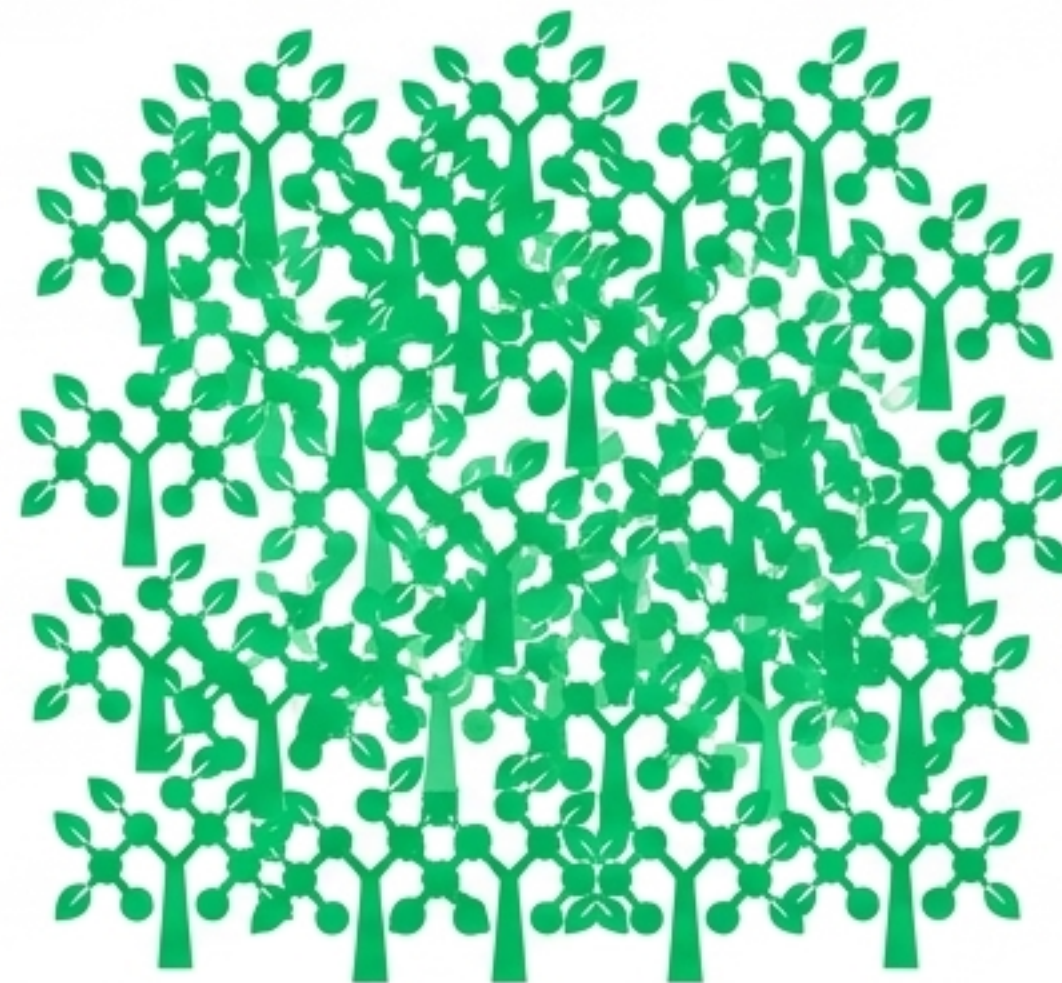
Phương sai của tập hợp được giảm mạnh hơn.

Nhìn chung, Bagging thường mang lại mô hình tổng quát hóa tốt hơn và là lựa chọn ưu tiên mặc định.

So sánh Cây đơn lẻ và Tập hợp túi hóa



Cây quyết định đơn lẻ

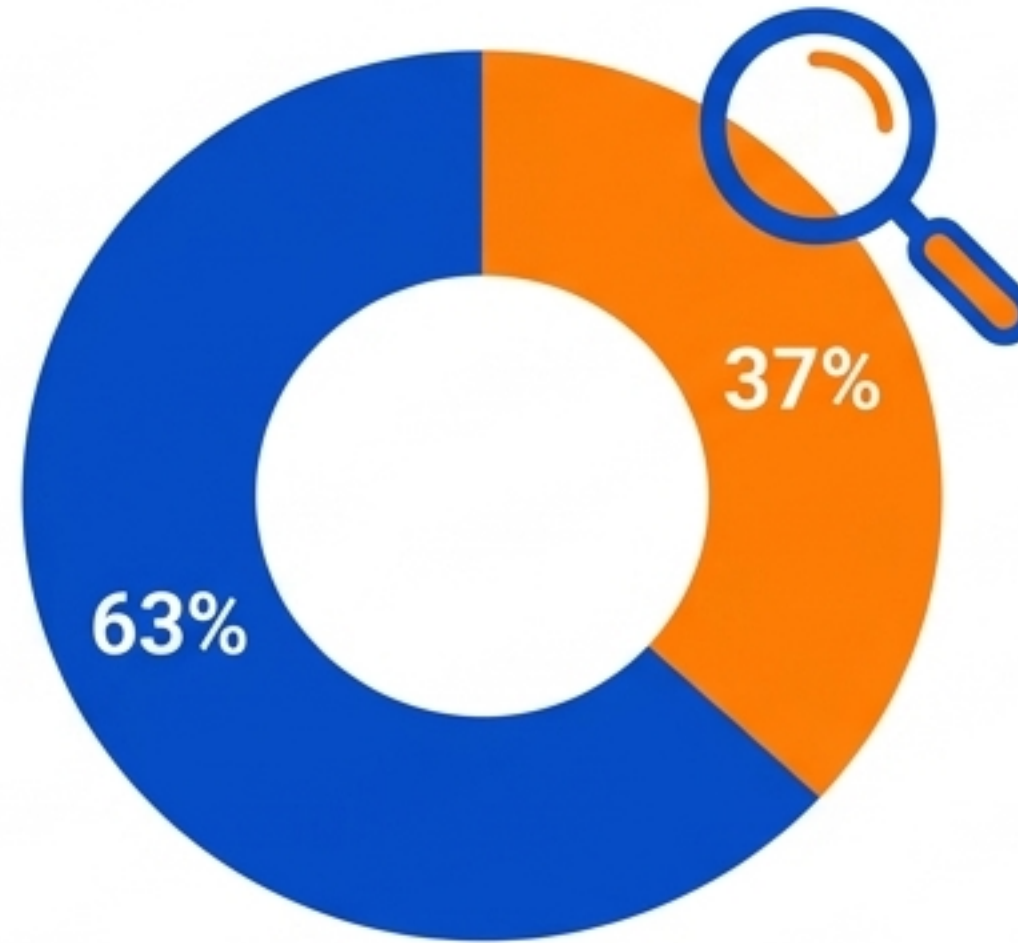


Tập hợp Bagging 500 cây

Hình 7-5. Trực quan hóa sự khác biệt giữa mô hình dự đoán đơn lẻ và sức mạnh tổng hợp của Bagging.

Đánh giá ngoài mẫu (Out-of-Bag Evaluation)

Việc lấy mẫu có hoàn lại (bootstrap) khiến một số trường hợp được lấy nhiều lần, số khác thì không.



63%

Số trường hợp trung bình mỗi bộ dữ liệu lấy.

37%

Các trường hợp ngoài mẫu (Out-of-Bag - OOB) không bao giờ được nhìn thấy.

Có thể dùng 37% OOB này để đánh giá tập hợp mà không cần tập xác thực riêng biệt.

Mã nguồn: Đánh giá OOB



Yêu cầu Scikit-Learn tính toán điểm OOB tự động bằng cách bật tham số `oob_score=True`.

```
1 bag_clf = BaggingClassifier(  
2     DecisionTreeClassifier(), n_estimators=500,  
3     oob_score=True, n_jobs=-1, random_state=42  
4 )  
5 bag_clf.fit(X_train, y_train)  
6  
7 # Xem điểm OOB  
8 print(bag_clf.oob_score_) # Output: 0.896  
9  
10 # Kiểm tra trên tập Test thực tế  
11 from sklearn.metrics import accuracy_score  
12 y_pred = bag_clf.predict(X_test)  
13 print(accuracy_score(y_test, y_pred)) # Output: 0.92
```

Random Patches và Random Subspaces

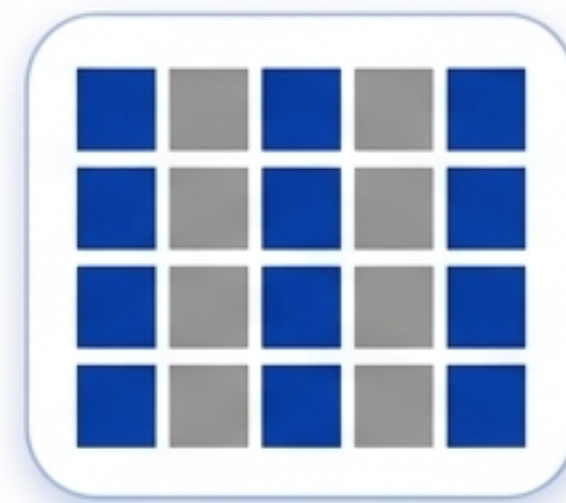
Lớp `BaggingClassifier` hỗ trợ lấy mẫu theo đặc trưng (features) thông qua `max_features` và `bootstrap_features`.



Random Patches

Lấy mẫu cả trường hợp huấn luyện và đặc trưng.

Rất hiệu quả cho dữ liệu nhiều chiều (chẳng hạn như hình ảnh).



Random Subspaces

Giữ toàn bộ trường hợp huấn luyện nhưng chỉ lấy mẫu ngẫu nhiên đặc trưng.

Đổi một chút độ lệch cao hơn để lấy phương sai thấp hơn.

Rừng ngẫu nhiên (Random Forests)

Tập hợp các cây quyết định, thường được huấn luyện bằng Bagging (với `max_samples` bằng toàn bộ kích thước tập).

Đưa thêm sự ngẫu nhiên: Thay vì tìm đặc trưng tốt nhất trong toàn bộ, thuật toán tìm đặc trưng tốt nhất trong một tập con ngẫu nhiên (mặc định là $\sqrt{n}$ đặc trưng).



Đa dạng cây lớn hơn → đổi độ lệch cao hơn lấy phương sai thấp hơn
→ mô hình tổng quát hóa tốt.

Mã nguồn: RandomForestClassifier



Sử dụng lớp

`RandomForestClassifier`
tiện lợi và tối ưu hơn so với
dùng `BaggingClassifier`
lồng
`DecisionTreeClassifier`.

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rnd_clf = RandomForestClassifier(
4     n_estimators=500,
5     max_leaf_nodes=16,
6     n_jobs=-1,
7     random_state=42
8 )
9 rnd_clf.fit(X_train, y_train)
10
11 y_pred_rf = rnd_clf.predict(X_test)
```

So sánh Bagging và Random Forest



Mã lệnh BaggingClassifier sau đây gần tương đương với RandomForestClassifier trước đó:

```
1 bag_clf = BaggingClassifier(  
2     DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),  
3     n_estimators=500,  
4     n_jobs=-1,  
5     random_state=42  
6 )  
7
```

Lớp RandomForestClassifier được tối ưu hiệu năng tốt hơn nhiều cho cây quyết định.

Cây ngẫu nhiên cực đại (Extra-Trees)

Sử dụng các ngưỡng ngẫu nhiên cho mỗi đặc trưng thay vì tìm ngưỡng chia cắt tốt nhất có thể. Tập hợp này được gọi là Extra-Trees (Extremely Randomized Trees).

Lợi ích:

- Tăng tốc độ huấn luyện đáng kể do không phải tốn kém thời gian tìm ngưỡng tốt nhất.
- Đổ thêm một chút độ lệch lấy phương sai thấp hơn nữa.



Cài đặt Extra-Trees trong Scikit-Learn



Trong DecisionTreeClassifier, đặt tham số `splitter="random"`, hoặc sử dụng tập hợp trực tiếp bằng ExtraTreesClassifier:

```
1 from sklearn.ensemble import ExtraTreesClassifier
2
3 extra_clf = ExtraTreesClassifier(
4     n_estimators=500,
5     max_leaf_nodes=16,
6     n_jobs=-1,
7     random_state=42
8 )
9 # API giống hệt với RandomForestClassifier
10
```

Tầm quan trọng của đặc trưng (Feature Importances)

Rừng ngẫu nhiên rất tuyệt vời để đo lường tầm quan trọng tương đối của từng đặc trưng.



Scikit-Learn tính toán mức độ mà các nút cây sử dụng một đặc trưng để làm giảm **độ không tinh khiết** (impurity).



Tính trung bình trên toàn bộ rừng.



Kết quả được **chuẩn hóa**: tổng độ quan trọng của tất cả các đặc trưng bằng 1.

Thuộc tính chứa kết quả trong Scikit-Learn: `feature_importances_`